

Opcenter RAG Project - Complete Interview Guide

A start-to-finish explanation of what was built, why each design choice was made, how the system is evaluated, and what is still required for production automation.

PROJECT	Opcenter manual-grounded RAG assistant
FOCUS	Architecture, decisions, evaluation, LLMOps and interview delivery
REPOSITORY	github.com/nagateja2004/op-center-bot

Contents

Use this guide to build a coherent story instead of memorizing disconnected tools.

Contents	2
1. How to present the project	4
The five-part structure	4
2. Problem, users and success criteria	5
The problem	5
Primary users	5
Success means	5
3. Complete architecture	6
4. Offline knowledge preparation	7
Why ingestion is separate	7
Pipeline	7
5. Online LangGraph workflow	8
6. Worked example: one complete request	9
Step-by-step	9
7. Retrieval: why hybrid search and reranking	10
Dense retrieval	10
Lexical retrieval	10
Weighted reciprocal-rank fusion	10
Cross-encoder reranking	10
8. Context expansion and deterministic compression	11
The failure mode	11
What the project does	11
9. Memory and follow-up questions	12
Type of memory	12
Why this design	12
10. LLM roles, calls, latency and cost	13
Cost controls	13
11. Asynchronous API and replica safety	14
Why two endpoints	14
Why Redis is necessary	14
Readiness	14
12. Grounding, citations, diagrams and safety	15
Grounding chain	15
Safe fallback	15
Diagrams	15
Security boundaries	15

13. Observability with LangSmith	16
Trace structure	16
What to inspect	16
Built-in metrics	16
14. Evaluation: dataset, rules and semantic judge	17
Golden dataset	17
What one test case contains	17
Deterministic evaluator	17
LLM-as-a-judge	17
15. Metrics and quality thresholds	18
What p50 and p95 mean	18
What semantic pass rate means	18
Artifacts	18
16. CI-gated evaluation and release	19
PR gate vs release gate	19
Why failed production traces point to the golden dataset	19
17. Current project status - say this honestly	20
18. Design decisions and trade-offs	21
19. Limitations and next improvements	22
Known limitations	22
Priority roadmap	22
20. Five-minute demo and final cheat sheet	23
Demo script	23
Ten facts to memorize	23

1. How to present the project

60-second answer

I built an asynchronous RAG assistant for Opcenter technical manuals. It ingests PDFs into structure-aware EvidenceUnits, retrieves with Chroma and BM25, fuses results with weighted reciprocal-rank fusion, expands neighboring context, reranks locally, grades evidence, and generates citation-grounded answers through a conditional LangGraph workflow. PostgreSQL provides thread memory, Redis coordinates requests and limits across two FastAPI replicas, Streamlit renders streamed answers, and LangSmith captures nested traces. I also built a 50-case golden dataset, deterministic checks, an LLM-as-a-judge, JSON/HTML reports, and CI quality gates. The delivery automation is implemented, while server secrets and staging configuration are still required for automatic deployment.

The five-part structure

- Problem: technical manuals are large, fragmented and hard to search accurately.
- Data plane: ingest manuals offline into structure-aware evidence and hybrid indexes.
- Online RAG: understand, retrieve, expand, rerank, grade, answer, verify and optionally diagram.
- Production engineering: asynchronous API, replica-safe Redis coordination, checkpoints, rate limits and tracing.
- Quality: golden cases, deterministic metrics, semantic judging, regression gates and human-reviewed feedback.

Interview rule

Say exactly what is implemented and what is configured. The repository contains the complete workflow code, but the latest automatic staging deployment did not run because GitHub SSH secrets are not configured. Do not claim that the full 50-case live release gate has passed.

2. Problem, users and success criteria

The problem

Opcenter documentation spans modeling, configuration, execution and product-specific manuals. A plain keyword search can miss paraphrases; a general LLM can invent unsupported details; and a naive vector-only chatbot can retrieve a relevant fragment while losing the warning, procedure step or table row that makes it correct. The project therefore optimizes for grounded technical answers rather than open-ended conversation.

Primary users

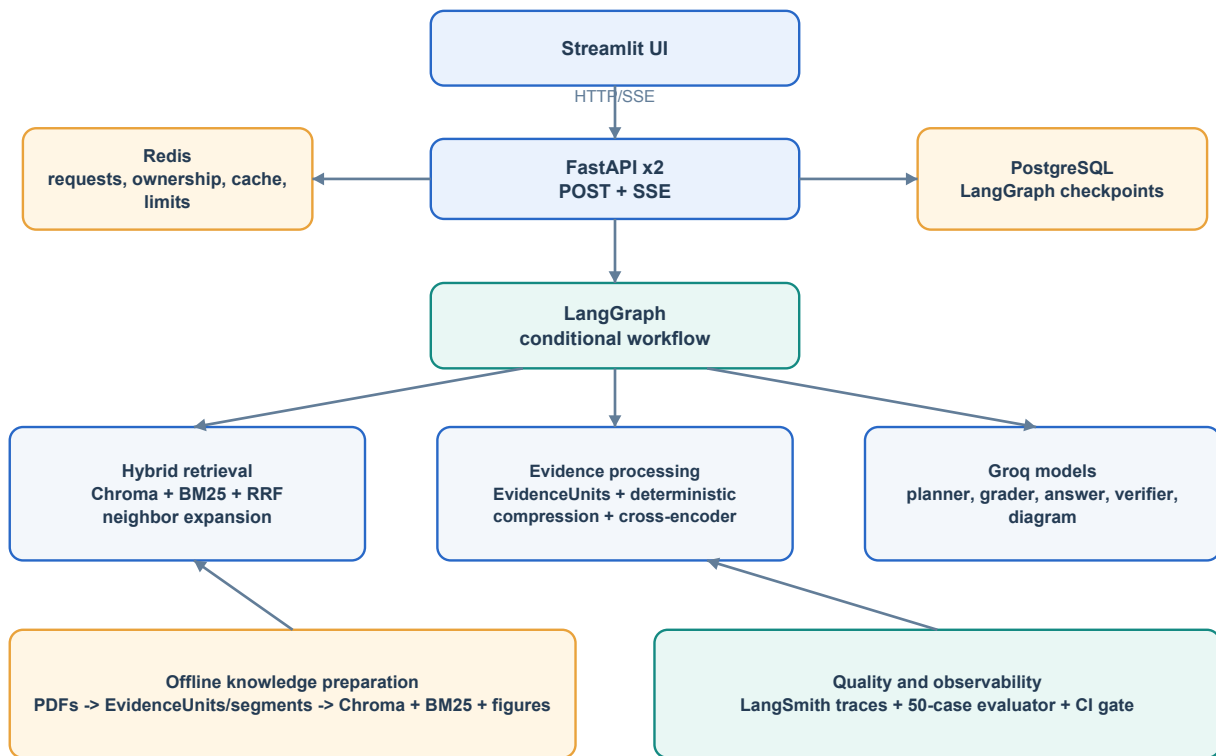
- Manufacturing engineers and implementation consultants looking for definitions, configuration steps and relationships.
- Support engineers troubleshooting an issue and needing a source-backed answer quickly.
- Developers and reviewers validating changes to the retrieval and prompting pipeline.

Success means

Dimension	Practical meaning
Grounding	Every substantive answer is supported by retrieved manual evidence.
Traceability	Citations map to stable returned source IDs and manual metadata.
Retrieval coverage	Direct wording, paraphrases, tables, procedures and comparisons are handled.
Safety	Insufficient and out-of-scope questions return limitations instead of invented answers.
Operability	Latency, failures, queues, tokens, cost and node traces can be inspected.
Regression control	A candidate is blocked when fixed thresholds or an accepted baseline regress.

3. Complete architecture

SYSTEM ARCHITECTURE - ONLINE, DATA AND QUALITY PLANES



Key boundary: ingestion is offline; chat requests only query existing indexes.

Layer	Technology	Why it is used
UI	Streamlit	Fast portfolio UI with session state and streamed rendering.
API	FastAPI	Typed asynchronous endpoints, dependency injection and SSE streaming.
Workflow	LangGraph	Explicit state, conditional routes, bounded retry and checkpoint integration.
Retrieval	Chroma + BM25 + weighted RRF	Combines semantic similarity with exact technical terminology.
Reranking	ms-marco-MiniLM cross-encoder	Scores the query and candidate together for better final ordering.
Generation	Role-specific Groq models	Different cost, latency and capability profiles per task.
State	PostgreSQL + Redis	Durable thread checkpoints plus distributed request coordination and limits.
Observability	LangSmith + built-in metrics	End-to-end traces and operational counters without a dashboard dependency.
Delivery	Docker Compose + GitHub Actions + SSH	Repeatable services and gated staging/production workflow.

4. Offline knowledge preparation

Why ingestion is separate

Indexes are built only when manuals or the schema change. Startup and chat requests never rebuild them. This keeps serving predictable, avoids duplicate work across replicas, and makes index versions auditable.

Pipeline

PDF manuals -> PyMuPDF extraction -> headings, text, tables, procedures, warnings and figures -> immutable EvidenceUnits -> smaller RetrievalSegments and search representations -> embeddings in Chroma + lexical BM25 index -> manifest and ingestion audit.

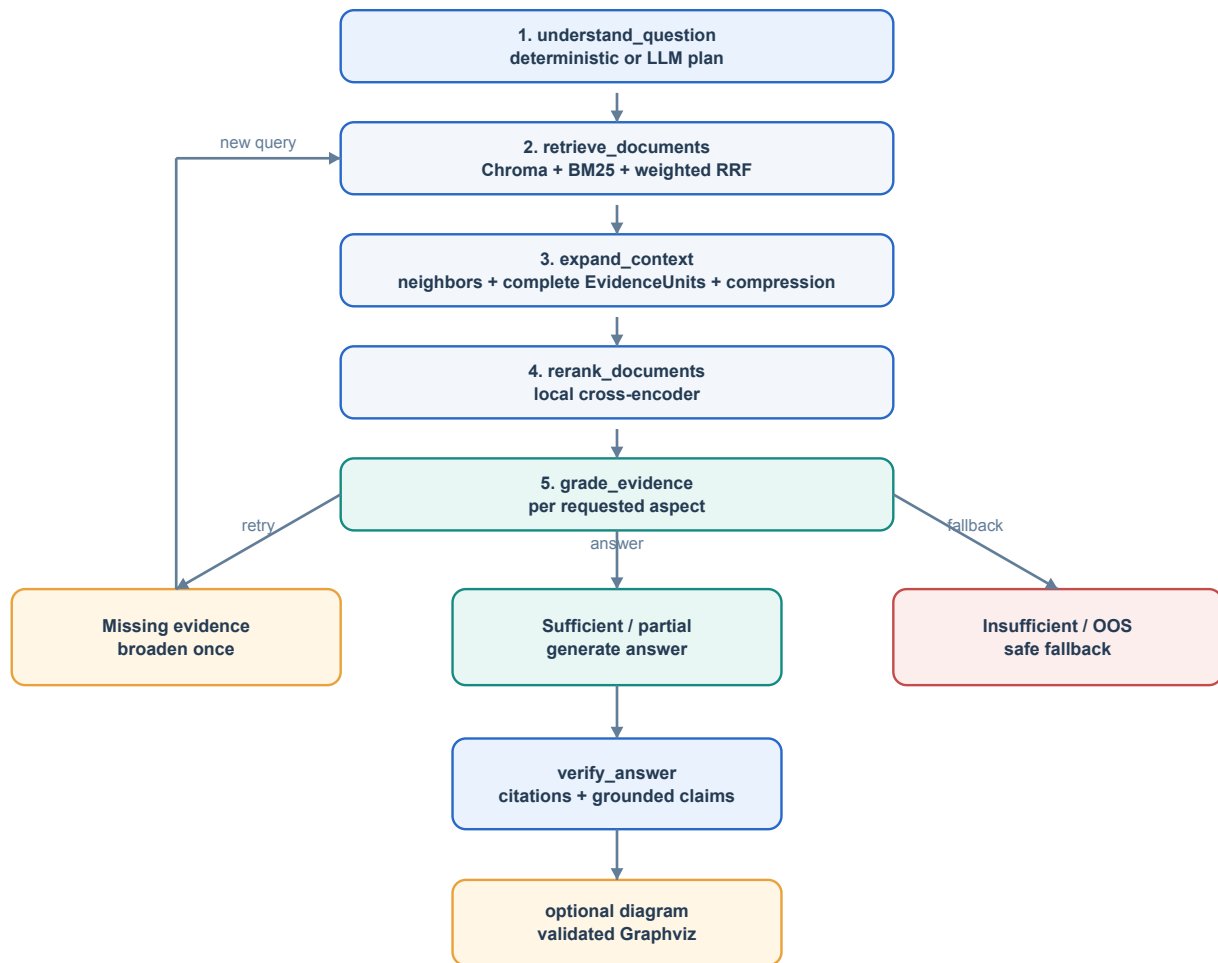
Artifact	Purpose
EvidenceUnits	Complete source units retained for reasoning and citations.
RetrievalSegments	Smaller searchable pieces that improve recall and fit embedding limits.
Search representations	Alternative searchable forms connected to the same evidence ID.
Heading/concept indexes	Support exact concepts, aliases and hierarchy-aware retrieval.
Chroma collection	Dense semantic search over embeddings.
BM25 index	Lexical search for exact product names, field names and acronyms.
Figure catalog	Maps cited manual pages to extractable original diagrams.
Manifest/audit	Verifies schema version, IDs and ingestion consistency.

Key design insight

Search small; answer from complete evidence. RetrievalSegments improve recall, but selected results are resolved back to immutable EvidenceUnits before generation.

5. Online LangGraph workflow

LANGGRAPH ONLINE REQUEST FLOW



Basic chat exits after step 1. Every node is timed and nested under one LangSmith trace.

Node	Responsibility	LLM?
understand_question	Detect basic chat, intent, aspects, follow-up context and diagram need.	0 or 1
retrieve_documents	Run hybrid dense/lexical retrieval and multi-query fusion.	No
expand_context	Add neighbors, resolve EvidenceUnits and build compressed views.	No
rerank_documents	Cross-encode the query against candidates.	No
grade_evidence	Judge support independently for each requested aspect.	1 per aspect/pass
broaden_query	Create one targeted query for missing evidence.	0 or 1
generate_answer	Write a grounded answer with source IDs.	0 or 1
verify_answer	Normalize citations and remove unsupported claims.	0 or 1
generate_diagram	Create source-grounded Graphviz when useful or requested.	0 or 1
generate_fallback	Return a safe limitation for insufficient evidence.	No

6. Worked example: one complete request

Example question

What is a Factory?

Step-by-step

- FastAPI accepts the message, assigns request/session/conversation/thread UUIDs and stores the pending request in Redis.
- The browser opens the SSE endpoint using the request ID. A backend replica atomically claims the pending request.
- LangGraph loads the PostgreSQL checkpoint for `conversation_id:thread_id`.
- `understand_question` recognizes a simple definition and uses deterministic planning, so no planner LLM call is needed.
- Chroma finds semantically similar segments; BM25 finds exact Factory terminology; weighted RRF combines rank positions.
- Neighbor expansion and EvidenceUnit resolution restore complete context. Deterministic compression selects relevant sentences without rewriting them.
- The cross-encoder reranks the strongest evidence. The grader marks the required aspect as supported.
- The answer model writes from the supplied evidence and uses [S1]-style citations. Verification validates those citations and removes unsupported wording.
- SSE sends progress events, answer chunks, sources, evidence status and an optional diagram back to Streamlit.
- The updated conversation state is checkpointed in PostgreSQL and the nested run is visible in LangSmith when tracing is enabled.

Follow-up example

If the next question is 'How is it related to resources?', the same thread checkpoint supplies the previous Factory context. The workflow resolves the follow-up into a standalone retrieval intent. A new conversation does not inherit it.

7. Retrieval: why hybrid search and reranking

Dense retrieval

Chroma stores MiniLM embeddings and is strong when the user's wording differs from the manual. It may retrieve 'manufacturing equipment' for a question that does not contain the exact word Resource.

Lexical retrieval

BM25 rewards exact token matches. This matters for acronyms, field names, class names, manual titles and rare product terminology that embedding similarity can blur.

Weighted reciprocal-rank fusion

RRF combines rank positions instead of trying to normalize incompatible Chroma and BM25 score scales. A document appearing high in either list receives credit, and documents supported by both lists rise further. The implementation uses an RRF constant of 60.

Cross-encoder reranking

The bi-encoder embedding stage is fast enough for broad candidate retrieval. The local cross-encoder then evaluates each query-document pair jointly, which is slower but more accurate. It only sees a bounded candidate set, and if unavailable the system safely retains the RRF order.

Best interview phrase

Dense retrieval improves semantic recall, BM25 protects exact technical recall, RRF combines them without score calibration, and the cross-encoder improves final precision.

8. Context expansion and deterministic compression

The failure mode

A retrieved fragment can omit the sentence before it, the condition after it, a later procedure step or the header of a table row. Passing every full page to the LLM, however, increases cost and distracts the model.

What the project does

- Adds previous and next RetrievalSegments around strong seeds.
- Resolves all selected segments to their complete parent EvidenceUnits.
- Keeps the original unit immutable for citation and verification.
- Creates a query/aspect-specific compressed view by selecting sentences, ordered procedure steps or table rows.
- Preserves nearby conditions and connected annotations when relevant.
- Stores selected indexes and character counts for auditability.

Approach	Advantages	Trade-off / decision
Deterministic compression (used)	No extra LLM call, repeatable, cheap, source text is preserved.	Lexical selection can miss subtle semantic relevance.
LLMChainExtractor	Can semantically extract relevant passages.	Adds latency/cost and may rewrite or omit evidence.
LLMChainFilter	Can reject irrelevant documents semantically.	Adds calls for multiple candidates and evaluator variance.

Answer if asked

We did not use LLMChainExtractor or LLMChainFilter. Compression comes after EvidenceUnit resolution and before grading/generation, using deterministic lexical selection. I would benchmark an LLM extractor only if recall errors justify its added latency and cost.

9. Memory and follow-up questions

Type of memory

The bot has durable short-term conversational memory, not long-term personal memory. LangGraph checkpoints are stored in PostgreSQL under a composite `conversation_id:thread_id` key. Redis binds a client thread to its originating anonymous session.

Identifier	Role
<code>session_id</code>	Identifies one browser session and owns its threads.
<code>conversation_id</code>	Groups a visible conversation.
<code>thread_id</code>	Selects the LangGraph checkpoint sequence.
<code>request_id</code>	Identifies one accepted request and its SSE stream.

Why this design

- A follow-up can use previous turns even when another backend replica handles it.
- A different browser session cannot claim the thread UUID.
- Starting a new conversation creates new IDs and prevents accidental context leakage.
- The system avoids permanent user profiling because it has no authenticated user identity or long-term memory store.

10. LLM roles, calls, latency and cost

Role	Default primary model	Typical calls
Planner	openai/gpt-oss-20b	0 or 1
Query broadening	openai/gpt-oss-20b	0 or 1
Evidence grader	openai/gpt-oss-20b	1 per aspect per retrieval pass
Answer	openai/gpt-oss-120b	0 or 1
Verifier	qwen/qwen3.6-27b	0 or 1
Diagram	openai/gpt-oss-20b	0 or 1
Evaluation judge	llama-3.1-8b-instant	1 per evaluated case + one error retry

How many LLM calls?

It is dynamic. A typical direct single-aspect RAG question uses three calls: grader, answer and verifier. An ambiguous question normally adds the planner. A retry adds broadening and another grading pass. A useful diagram adds one. Basic chat uses zero RAG LLM calls. Retrieval, expansion, deterministic compression, reranking and citation parsing use no LLM.

Cost controls

- Deterministic planning for simple definitions avoids unnecessary planner calls.
- Role-specific models avoid using the largest model for every task.
- Input token budgets are enforced per role.
- Each role has one primary and at most one fallback model.
- Redis enforces per-model concurrency, requests per minute, tokens per minute, queue depth and wait time.
- Token usage is logged; LangSmith can expose trace-level cost when provider/model pricing metadata is available.

11. Asynchronous API and replica safety

Why two endpoints

POST /v1/chat validates the request and returns HTTP 202 quickly. GET /v1/chat/{request_id}/stream then emits server-sent events. This separates request acceptance from a long-running LLM workflow and gives the UI progress feedback.

SSE event	Meaning
progress	A LangGraph node completed.
answer	A piece of answer text is available.
complete	Final answer, sources, evidence status, figures and diagram.
error	A sanitized client-facing failure.

Why Redis is necessary

With two backend replicas, an in-memory pending-request map would fail whenever POST reached one replica and the SSE request reached another. Redis stores and atomically pops the request, tracks thread ownership, final-answer cache entries, Groq admission and request status, so any replica can continue safely.

Readiness

The /ready endpoint verifies the compiled graph, PostgreSQL, Redis, Chroma, BM25, EvidenceUnit store, embedding model and reranker. /health only proves that the process is alive.

12. Grounding, citations, diagrams and safety

Grounding chain

Stable evidence IDs survive ingestion, Chroma/BM25 retrieval, RRF, reranking and response construction. The answer prompt allows only supplied EvidenceUnits and requires [S#] citations. Verification works from cited evidence only, normalizes citation IDs and removes unsupported claims.

Safe fallback

The grader can return sufficient, partial, retry, in-scope insufficient or out-of-scope behavior. Retrieval is broadened only once. If support is still missing, the deterministic fallback explains the limitation rather than asking the answer model to guess.

Diagrams

- Original manual figures are preferred when a cited page has a usable extracted image.
- Generated diagrams use Graphviz DOT for hierarchy, relationship, process, decision or architecture views.
- Generation is allowed only for a grounded answer and when requested or useful.
- DOT is parsed and validated deterministically; source citations must remain traceable.
- Diagram failure does not discard an otherwise grounded text answer.

Security boundaries

- Never commit .env, API keys, passwords, manuals or generated indexes.
- Evaluation evidence is returned only with a constant-time checked X-Evaluation-Token.
- Evaluation context is bounded to eight cited sources and 12,000 characters total.
- Request size, message length, CORS, TTLs and queue depths are bounded.
- Client errors and health responses do not expose internal stack traces.

13. Observability with LangSmith

Trace structure

Each request creates one root trace named `opcenter_chat`, tagged `opcenter-rag` and `streaming-api`. LangGraph nodes and LangChain/Groq calls are nested beneath it. Searchable metadata includes request, session, conversation, client thread, diagram choices, evaluation flag and deployment environment.

What to inspect

Question	Trace evidence
Why was the answer wrong?	Retrieved evidence, grader decision, prompt inputs and verifier output.
Where was time spent?	Root latency and per-node/model durations.
Was the model retried?	Primary/fallback model spans and errors.
What did it cost?	Input/output tokens and trace cost where pricing is configured.
Is production healthy?	Error rate, p50/p95 latency, cost and feedback aggregates.

Built-in metrics

The application also exposes process-local text metrics for HTTP latency, LangGraph node duration, Groq requests/tokens, local inference and queue depth. No external Prometheus server is required by the project, although the output format is scrape-compatible.

Data policy

LangSmith may capture questions, outputs and retrieved manual evidence. Tracing is optional and must remain disabled when deployment policy does not allow that data to leave the environment.

14. Evaluation: dataset, rules and semantic judge

Golden dataset

Category	Cases	Purpose
Direct	7	Definitions and explicit concepts
Indirect	8	Paraphrase and semantic retrieval
Follow-up	5	Thread memory and context
Procedure	5	Ordered grounded steps
Table/field	5	Structured row/field retrieval
Comparison	4	Coverage of all requested concepts
Unsupported	3	In-scope but insufficient evidence
Irrelevant	3	Out-of-scope routing
Execution Electronics	5	Product-specific routing
Execution Discrete	5	Product-specific routing

What one test case contains

A case contains an input question, optional context question, expected evidence status, expected terms or required terms, expected manual labels and optional diagram expectation. The evaluator adds the actual answer, returned sources, evidence state, retrieved evaluation context, latency and judge output.

Deterministic evaluator

Code checks term coverage, manual routing, evidence status, citation/source ID consistency, diagram state, API errors and latency. It is cheap and repeatable but can be brittle when a correct answer uses different wording.

LLM-as-a-judge

A separate Groq judge runs at temperature 0 with JSON output. It scores relevance, correctness, faithfulness, completeness and clarity from 1 to 5 using the gold constraints and protected retrieved evidence. A case passes when the mean is at least 4.0 and both correctness and faithfulness are at least 4.0. One invalid response retry is allowed.

15. Metrics and quality thresholds

Metric	Default gate
Overall deterministic pass rate	>= 85%
Answer-term accuracy	>= 85%
Manual-routing accuracy	>= 90%
Evidence-status accuracy	>= 90%
Citation-ID accuracy	>= 95%
P95 end-to-end latency	<= 75 seconds
Evaluation API errors	0
Semantic-quality average	>= 4.0 / 5
Semantic case pass rate	>= 85%
Judge errors	0
Accepted-baseline regression	Maximum normalized drop 3 percentage points

What p50 and p95 mean

p50 is the median: half the requests finish faster and half slower. p95 is a tail-latency measure: 95% of measured requests finish at or below that value, while the slowest 5% are worse. p95 exposes occasional queue, model or retrieval delays that an average can hide.

What semantic pass rate means

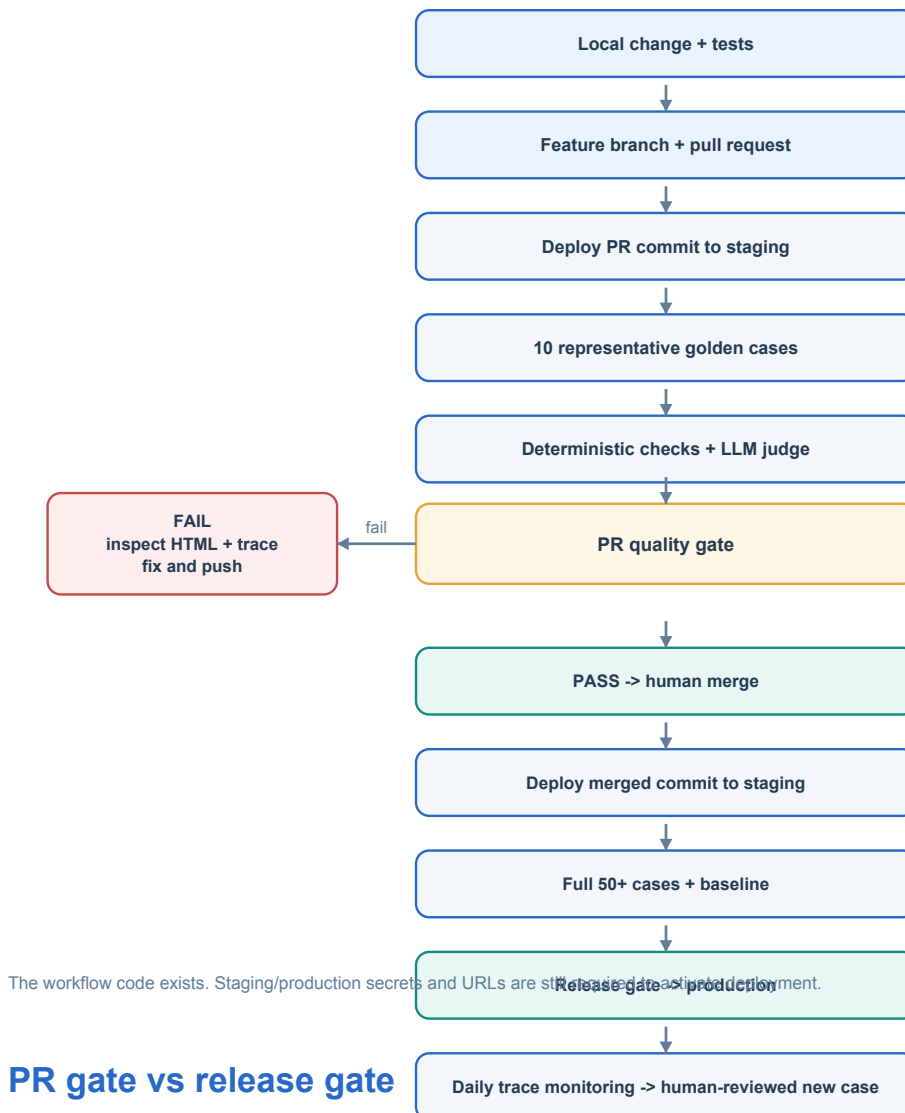
It is the percentage of cases whose semantic judge result passes the case rule. For example, 9 passing cases out of 10 gives 90%. It is different from the average semantic score because a high average can hide several individual failures.

Artifacts

- Machine-readable JSON with aggregate and per-case results.
- Human-readable HTML dashboard with expected output, actual answer, sources, checks, latency and judge reason.
- A non-zero process exit when the gate fails, allowing GitHub Actions to block promotion.

16. CI-gated evaluation and release

CI-GATED RELEASE FLOW



PR gate vs release gate

Gate	Dataset	Purpose
PR gate	10 representative questions - one from each category	Fast feedback before merge; avoids running all costly live judge calls on every commit.
Release gate	Full 50+ cases on staging	Validate the merged commit and compare with the accepted baseline before production.
Nightly monitor	Recent production LangSmith root traces	Detect error, tail-latency, cost or feedback regressions.

Why failed production traces point to the golden dataset

The production trace is not added automatically. A human first verifies the question, manual evidence and expected behavior. Only then is a sanitized regression case added through a pull request. This converts a real failure into permanent coverage without allowing noisy or malicious traffic to poison the dataset.

17. Current project status - say this honestly

Capability	Status as of 2026-08-03
Core asynchronous LangGraph RAG	Implemented in the repository.
Hybrid retrieval, EvidenceUnits and deterministic compression	Implemented with unit/integration coverage.
PostgreSQL memory, Redis coordination and two backend replicas	Implemented in Docker Compose.
LangSmith tracing configuration	Implemented and locally verified; production monitoring requires credentials.
50-case versioned golden dataset	Implemented.
Deterministic + semantic evaluator and reports	Implemented.
GitHub evaluation contract	Passed on main after merge commit f122c8c.
Latest stored live smoke evaluation	Only 3 cases; 2 passed; gate failed. This is not a full 50-case benchmark.
Automatic staging/production delivery	Workflow implemented, but the run failed because staging SSH secrets/variables are not configured.
Production release of latest main	Not automatically performed by the new workflow.
Latest smoke-run evidence 3 cases completed with 0 API errors and semantic average 4.9/5 for the two judged cases, but deterministic pass rate was 66.7%, p95 latency was 130.5 seconds, one judge error occurred, and the gate failed. Present this as a diagnostic smoke run that identified calibration and latency work - not as a production quality result.	
Strong honest answer The production-oriented mechanisms are implemented, but I would not call the latest commit fully production-validated until the staging environment is configured and the complete 50-case live gate passes against a human-approved baseline.	

18. Design decisions and trade-offs

Decision	Why	Cost / limitation
RAG instead of fine-tuning	Manuals change and citations need current source evidence.	Retrieval quality becomes a critical dependency.
Hybrid retrieval	Protects semantic and exact-term recall.	More indexes and fusion logic to operate.
EvidenceUnits + segments	Search precision without losing complete source context.	More complex ingestion schema and ID validation.
Deterministic compression	Repeatable, source-preserving and zero LLM cost.	May miss subtle semantic passages.
Role-specific LLMs	Balance quality, latency and cost.	More model policies and failure modes.
LangGraph	Explicit branching, retry and checkpoint state.	More structure than a simple chain.
PostgreSQL checkpoints	Durable, multi-replica follow-up memory.	Database lifecycle and retention must be managed.
Redis coordination	Replica-safe ownership, queues, cache and rate limits.	Additional service dependency.
LLM judge + deterministic rules	Combines semantic understanding with repeatability.	Judge cost/variance and human calibration are still required.
10 PR / 50 release cases	Fast feedback plus broader release confidence.	A small PR subset can miss category-specific regressions.

19. Limitations and next improvements

Known limitations

- Scanned PDFs require OCR before ingestion.
- Anonymous sessions are not identity-based access control.
- Conversation retention and deletion policies still need production governance.
- Built-in metrics are process-local; a multi-replica dashboard needs aggregation.
- LLM judge scores must be calibrated against human labels before becoming a mandatory merge control.
- The current full live 50-case evaluation and accepted baseline have not yet been completed.
- Automatic staging/production delivery requires environment secrets, URLs and server directories.

Priority roadmap

- Configure an isolated staging server and GitHub environments without exposing production manuals or keys.
- Run the 50-case live evaluation, inspect every failure and establish a human-approved baseline.
- Optimize p95 by tracing model and node time, tightening prompts, caching safe results and profiling queues.
- Add retrieval recall metrics with expected evidence IDs/pages for more cases.
- Calibrate the semantic judge against a manually scored validation set and measure judge agreement.
- Add authentication and role-based document access if the system becomes multi-tenant.

20. Five-minute demo and final cheat sheet

Demo script

- Open the Streamlit UI and ask one direct definition question. Point out streamed progress and citations.
- Ask a related follow-up in the same conversation. Explain PostgreSQL checkpoints and Redis thread ownership.
- Ask an out-of-scope question. Show the safe fallback instead of hallucination.
- Open the matching LangSmith trace. Walk through retrieval, grade, generation, verification and latency spans.
- Open the evaluation HTML report. Explain deterministic checks, semantic scores and why a gate can fail.
- Open GitHub Actions and distinguish the passing evaluation contract from the currently unconfigured delivery job.

Ten facts to memorize

Prompt	Answer
Memory	Durable short-term thread memory in PostgreSQL; no long-term profile.
Compression	Deterministic after EvidenceUnit resolution; no LLMChainExtractor/filter.
Typical LLM calls	3: grader, answer, verifier.
Retrieval	Chroma + BM25 -> weighted RRF -> neighbor expansion -> cross-encoder.
Retry	At most one query-broadening retry.
API	POST accepts; GET streams SSE.
Scale	Two FastAPI replicas coordinated by Redis.
Dataset	50 cases in 10 categories.
Semantic judge	Five dimensions; mean/correctness/faithfulness must be ≥ 4 .
Current status	Main updated; contract passed; full live gate and automatic deployment still pending configuration.

Closing statement

The strongest part of this project is not only that it answers manual questions. It makes retrieval, evidence, model decisions, quality and release behavior inspectable and testable - while clearly separating implemented controls from unvalidated production claims.